



دانشگاه علم و صنعت ایران - دانشکده مهندسی کامپیوتر

پروژه درس طراحی الگوریتم

موضوع: مسیریابی هوشمند شهری

استاد: دکتر آرمان سپهر

تهیه کنندگان:

ایلیا صادق احمدی

محمد منتظری

گزارش کار پروژه

سیستم مسیریابی هوشمند برای خودروهای خدماتی شهری

پیاده‌سازی الگوریتم‌های *Dijkstra* و *Bellman-Ford* به همراه مسیریابی چندمقصدی (*TSP*)

۱. مقدمه

این گزارش، مستندِ کاره‌ی پروژه‌ی «سیستم مسیریابی هوشمند برای خودروهای خدماتی شهری» است که طبق صورت پروژه، در چهار فاز پیاده‌سازی شده است: ساخت گراف جهت‌دار وزن‌دار شهر، پیدا کردن کوتاه‌ترین مسیر با الگوریتم دایکسترا، پشتیبانی از وزن‌های منفی و تشخیص دور منفی با الگوریتم بلمن-فورد، و در نهایت مسیریابی بین چند مقصد با یک نسخه‌ی ساده‌شده از مسئله‌ی فروشنده‌ی دوره‌گرد (*TSP*).

تمام کدها با زبان Python در فایل `smart_routing.py` نوشته شده‌اند و برای آزمایش، سه فایل نقشه‌ی نمونه (`sample_map.txt`، `sample_map_negative_cycle.txt` و `sample_map_negative_cycle.txt`) تهیه شده که در ادامه‌ی این گزارش به تفصیل معرفی می‌شوند.

۲. ساختار داده و طراحی گراف

شهر به صورت یک گراف جهت‌دار و وزن‌دار مدل شده و برای نگهداری آن از ساختار داده‌ی لیست مجاورت (**Adjacency List**) استفاده شده، نه ماتریس مجاورت. دلیل این انتخاب آن است که نقشه‌ی واقعی شهر معمولاً یک گراف «تُنک» (*sparse*) است؛ یعنی هر تقاطع تنها به تعداد محدودی تقاطع همسایه وصل است، نه به همه‌ی تقاطع‌های شهر. لیست مجاورت هم از نظر مصرف حافظه (به جای $O(V^2)$ ، مصرف حافظه‌ی $O(V + E)$) و هم از نظر سرعت پیمایش یال‌های خروجی هر گره، گزینه‌ی مناسب‌تری نسبت به ماتریس مجاورت است.

هر یال جهت‌دار است؛ به این معنا که برای خیابان‌های دوطرفه باید دو یال جدا (یک رفت و یک برگشت) در فایل ورودی تعریف شود. هزینه‌ی عبور از هر یال طبق فرمول زیر که در صورت پروژه آمده محاسبه می‌شود:

$$\text{cost}(u, v) = d(u, v) \times t(u, v) \times w(u, v) + \text{delay}[v]$$

که در آن d فاصله‌ی بین دو تقاطع، t ضریب ترافیک، w ضریب آب‌وهوا/وضعیت جاده و $\text{delay}[v]$ تاخیر ثابت تقاطع مقصد است. این فرمول در متد `add_edge` کلاس `Graph` پیاده‌سازی شده و هزینه‌ی هر یال از همان ابتدا، هنگام ساخت گراف، محاسبه و ذخیره می‌شود.

۳. فاز اول: خواندن ورودی و ساخت گراف

هدف این فاز، خواندن کامل نقشه از یک فایل متنی ساده و ساخت گراف قابل پردازش است. فرمت فایل ورودی به شکل زیر تعریف شده:

node_id node_name -> این خط V بار تکرار می‌شود

u v d t w delay -> این خط E بار تکرار می‌شود

که V و E به ترتیب تعداد گره‌ها و تعداد یال‌ها هستند. خواندن فایل در تابع `read_graph_from_file` انجام می‌شود و خط‌های خالی نادیده گرفته می‌شوند تا تابع نسبت به فاصله‌های اضافه در فایل حساس نباشد. برای مثال، محتوای فایل `sample_map.txt` که برای این پروژه ساخته شده، شامل ۶ تقاطع (A تا F) و ۱۸ یال جهت‌دار است (۹ خیابان دوطرفه):

6 18

A Intersection_A B Intersection_B C Intersection_C

D Intersection_D E Intersection_E F Intersection_F

A B 4 1.2 1.0 0 B A 4 1.2 1.0 0

A C 2 1.0 1.1 0 C A 2 1.0 1.1 0

...

پس از اجرای این فاز روی فایل نمونه، خروجی برنامه ساخته‌شدن گراف را این‌گونه تایید می‌کند:

گراف ساخته شد: <Graph: ۶> گره، 18 یال

۴. فاز دوم: کوتاه‌ترین مسیر با الگوریتم دایکسترا

الگوریتم Dijkstra در تابع `dijkstra` با استفاده از یک صف اولویت مبتنی بر مین‌هیپ (`heapq`) پیاده‌سازی شده است. در هر مرحله، گره‌ای که فعلاً کمترین هزینه را دارد از صف خارج و یال‌های خروجی آن ریلکس (`relax`) می‌شوند. این الگوریتم تنها زمانی جواب درست می‌دهد که هیچ یالی وزن منفی نداشته باشد. با توجه به استفاده از صف اولویت، پیچیدگی زمانی آن به $O((V + E) \log V)$ می‌رسد.

پس از پیدا شدن کوتاه‌ترین هزینه، مسیر واقعی با تابع `reconstruct_path` و با دنبال کردن آرایه‌ی `parent` از مقصد به مبدا بازسازی می‌شود.

نتیجه‌ی اجرای این فاز روی فایل `sample_map.txt` برای یافتن کوتاه‌ترین مسیر از تقاطع A تا تقاطع F به شرح زیر است:

کوتاه‌ترین هزینه از A تا F: ۱۵.۲

مسیر: A -> C -> E -> F

یعنی با وجود آن‌که یال مستقیم $E \rightarrow F$ هزینه‌ی کمی دارد، مسیر $A \rightarrow C \rightarrow E \rightarrow F$ در مجموع ارزان‌تر از سایر مسیرهای ممکن (مانند عبور از B یا D) تشخیص داده شده است.

۵. فاز سوم: وزن منفی و الگوریتم بلمن-فورد

چون دایکسترا در حضور یال منفی جواب نادرست می‌دهد، تابع `shortest_path` ابتدا با متد `has_negative_edge` گراف را بررسی می‌کند و در صورت وجود حتی یک یال منفی، به جای دایکسترا از

الگوریتم Bellman-Ford (تابع bellman_ford) استفاده می‌کند. این الگوریتم در $V-1$ دور، تمام یال‌های گراف را ریلکس می‌کند و سپس با یک دور اضافه، وجود دور با وزن منفی را بررسی می‌کند. پیچیدگی زمانی آن $O(V \cdot E)$ است.

برای آزمایش این فاز، فایل `sample_map_negative.txt` ساخته شده که همان نقشه‌ی ۶ تقاطعی است با یک یال اضافی منفی از D به F با هزینه‌ی -6 (در مجموع ۱۹ یال). نتیجه‌ی اجرای کوتاه‌ترین مسیر از A تا F روی این فایل:

(با یال منفی) کوتاه‌ترین هزینه از A تا F: ۴.۶۹۹۹۹۹۹۹۹۹۹۹۹۹

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow F$: مسیر

همان‌طور که دیده می‌شود، وجود یال منفی $D \rightarrow F$ باعث شده مسیر بهینه به کلی نسبت به فاز قبل تغییر کند و هزینه‌ی کل به شدت کاهش یابد؛ این دقیقاً رفتاری است که از بلمن-فوردر در حضور وزن منفی انتظار می‌رود.

برای بررسی تشخیص دور منفی، فایل جداگانه‌ی `sample_map_negative_cycle.txt` با یک گراف بسیار ساده‌ی سه‌گره‌ی ساخته شده: $A \rightarrow B \rightarrow C \rightarrow A$ که مجموع وزن این دور برابر $1 + 1 + (-5) = -3$ است، یعنی، یک دور منفی واقعی، اجرای بلمن-فوردر روی این گراف به‌درستی خطای زیر را تولید می‌کند:

تشخیص دور منفی به درستی انجام شد ->

یک دور با وزن منفی در گراف پیدا شد (روی یال $A \rightarrow B$).

در این حالت محاسبه‌ی کوتاه‌ترین مسیر معنا ندارد.

این نتیجه نشان می‌دهد بخش تشخیص دور منفی الگوریتم (استثنای NegativeCycleError) به درستی کار می‌کند و به جای بازگرداندن یک جواب گمراه‌کننده، وجود مشکل را صراحتاً اعلام می‌کند.

۶. فاز چهارم: مسیریابی چند مقصدی (نزدیک ترین همسایه)

این فاز نسخه‌ی ساده‌شده‌ای از مسئله‌ی فروشنده‌ی دوره‌گرد (TSP) را حل می‌کند: شروع از یک تقاطع، سر زدن به چند مقصد مشخص و بازگشت نهایی به همان تقاطع شروع. چون TSP یک مسئله‌ی NP-Hard است، به جای جواب دقیق از یک الگوریتم حریصانه (greedy) به نام «نزدیک‌ترین همسایه» (Nearest Neighbor Heuristic) در تابع `nearest_neighbor_route` استفاده شده است: در هر قدم، از میان مقصدهای باقی‌مانده، نزدیک‌ترین آن‌ها از نظر هزینه‌ی واقعی کوتاه‌ترین مسیر در گراف (نه فاصله‌ی مستقیم) با فراخوانی تابع `shortest_path` انتخاب می‌شود.

در آزمایش این فاز، مسیریابی از تقاطع A با مقصدهای {C, E, F} روی نقشه‌ی sample_map.txt اجرا شده که نتیجه‌ی آن به شرح زیر است:

ترتيب توقفها: $A \rightarrow C \rightarrow E \rightarrow F \rightarrow A$

هزینه‌ی کل مسیر: 30.4

مسیر کامل گره به گره در هر بخش:

بخش 1: $A \rightarrow C$

بخش 2: $C \rightarrow E$

بخش 3: $E \rightarrow F$

بخش 4: $F \rightarrow E \rightarrow C \rightarrow A$

قابل توجه است که در بازگشت از F به A، الگوریتم به جای استفاده از یال مستقیم و پرهزینه $F \rightarrow D \rightarrow \dots$ ، مسیر $F \rightarrow E \rightarrow C \rightarrow A$ را انتخاب کرده که با احتساب فرمول هزینه، ارزان تر تشخیص داده شده است.

۷. فایل های تست تهیه شده

برای پوشش کامل سناریوهای پروژه، سه فایل نقشه‌ی نمونه ساخته و ضمیمه شده‌اند که در جدول زیر معرفی می‌شوند:

نام فایل	مشخصات	هدف از ساخت
sample_map.txt	6 گره، 18 یال (بدون وزن منفی)	آزمایش پایه‌ی فاز ۱ و ۲؛ اجرای الگوریتم دایکسترا و مسیریابی چندمقصودی فاز ۴
sample_map_negative.txt	6 گره، 19 یال (یک یال منفی، بدون دور منفی)	آزمایش فاز ۳؛ بررسی درستی بلمن-فورد در حضور وزن منفی بدون دور منفی
sample_map_negative_cycle.txt	3 گره، 3 یال (دور منفی واقعی)	آزمایش فاز ۳؛ بررسی تشخیص دور با وزن منفی و صدور خطا

۸. تحلیل پیچیدگی زمانی و مکانی

جدول زیر پیچیدگی زمانی و مکانی الگوریتم‌های استفاده شده در هر فاز را خلاصه می‌کند:

پیچیدگی مکانی	پیچیدگی زمانی	کاربرد در پروژه	الگوریتم
$O(V+E)$	$O((V+E) \log V)$	کوتاه‌ترین مسیر برای وزن‌های نامنفی (فاز ۲)	دایکسترا (Dijkstra)
$O(V)$	$O(V \cdot E)$	پشتیبانی وزن منفی و تشخیص دور منفی (فاز ۳)	بلمن-فورد (Bellman-Ford)
به هزینه‌ی SP وابسته است	$O(K^2 \cdot SP)$	مسیریابی چندمقصودی حریصانه (فاز ۴)	نزدیک‌ترین همسایه (TSP)

که در آن K تعداد مقصدهای باقی مانده و SP هزینه‌ی یک بار اجرای کامل الگوریتم کوتاه‌ترین مسیر بین دو گره است. اگر گراف یال منفی نداشته باشد، SP برابر هزینه‌ی دایکسترا و در غیر این صورت برابر هزینه‌ی بلمن-فورد خواهد بود.

۹. جمع‌بندی

در این پروژه یک سیستم مسیریابی کامل برای شبیه‌سازی حرکت خودروهای خدماتی در شهر پیاده‌سازی شد که شامل ساخت گراف از فایل ورودی، محاسبه‌ی کوتاه‌ترین مسیر با دو الگوریتم مکمل ($Dijkstra$ برای حالت معمول و $Bellman-Ford$ برای حالت وزن منفی و تشخیص دور منفی) و در نهایت مسیریابی بین چند مقصد با یک الگوریتم حریصانه است. نتایج اجرا روی هر سه فایل نمونه نشان داد که خروجی‌ها در همه‌ی سناریوها (بدون وزن منفی، با وزن منفی بدون دور، و با دور منفی) مطابق انتظار و منطقی هستند.